MODULE 04 · DOMAIN 5 - MODEL SELECTION AND OPTIMISATION

# SDKs, REST and Websockets

What the official SDKs do over plain HTTP, when an abstraction layer earns its keep, and where websockets belong.

DOMAIN 5 of 8 EXAM WEIGHT 16.8% MODULE 4 of 36

admin@sthithapragna.com

An SDK is a careful wrapper around plain HTTP. Know what it does, so you know what you would have to rebuild.

# The same request, as raw HTTP

```
curl https://api.anthropic.com/v1/messages \
  -H "Authorization: Bearer $ANTHROPIC_API_KEY" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-5-5", "max_tokens": 1024,
       "messages": [{"role": "user", "content": "When is my rent due?"}]}'
```

2 The key, from the environment. Bearer is the primary header; x-api-key still works.

3 A required API version header.

5-6 The JSON body: the same fields the SDK sends for you.

THE TAKEAWAY

This is all an SDK call becomes on the wire: a POST with headers and a JSON body.

# What the official SDK does for you

| JOB | WHAT THE SDK HANDLES |
| --- | --- |
| Authentication | Reads the key from the environment, sets the headers |
| Retries | Connection errors, 429 and 5xx: two retries by default, exponential backoff, honours retry-after |
| Timeouts | Refuses non-streaming requests expected to run past 10 minutes |
| Streaming | Parses server-sent events into typed events and text |
| Types and errors | Typed requests and responses; one exception class per error |
| Platforms | Client classes for Bedrock, Google Cloud, Foundry, Claude Platform on AWS |

ON THE EXAM

Each row is code you would otherwise write, test and maintain yourself.

# Tuning the SDK instead of replacing it

```python
client = anthropic.Anthropic(max_retries=4, timeout=60.0)

with client.messages.stream(
    model="claude-sonnet-5-5", max_tokens=8000,
    system=SYSTEM_PROMPT, messages=history,
) as stream:
    for text in stream.text_stream:
        send_to_tenant(text)
    final = stream.get_final_message()
```

1 Retries and timeout are settings, not code you write.

7 The helper turns streaming events into plain text chunks.

9 The complete message, with stop reason and usage, at the end.

THE TIEBREAKER

Need different retry or timeout behaviour? Change a setting. That is almost never a reason to drop the SDK.

# Official SDK or raw HTTP?

USE THE SDK WHEN

## The normal case

- An official SDK exists for your language
- You want retries, streaming and types
- Several services share one client
- Nothing forbids the dependency

USE RAW HTTP WHEN

## There is a stated reason

- No SDK for your language
- A locked-down script that cannot add packages
- Debugging exactly what goes on the wire
- Then rebuild retries yourself

THE TIEBREAKER

Default to the SDK. Drop to raw HTTP only for a reason the scenario actually states.

# A multi-provider abstraction layer?

- **Anthropic's advice:** start with the APIs directly; many patterns take only a few lines of code.
- **Frameworks add layers** that can obscure the prompts and responses you need to debug.
- **Lowest common denominator:** provider-specific features such as citations or caching may be hidden or lag.
- **It earns its keep** when a real requirement exists: several providers on the roadmap, or contractual portability.

> ON THE EXAM
>
> Abstraction is a cost. Pay it when a requirement asks for it, not for hypothetical flexibility.

# Copperlet's abstraction debate

THE PROPOSAL

## Third-party multi-provider layer

- A new dependency in every service
- Generic features only
- Prompts harder to see when debugging
- No roadmap item needs another provider

THE DECISION

## A thin seam of Copperlet's own

- One client module calls Claude
- Model and platform come from config
- Harwood's calls use the Bedrock client
- Citations and caching stay available

THE TIEBREAKER

One customer on Bedrock needs a different client class, not a framework. A thin internal seam covers it.

Five ways to move data. Pick by **who talks, and how often**.

# Five communication patterns

| PATTERN | HOW IT WORKS | TYPICAL USE |
|---|---|---|
| Request-response | Client asks, server answers once | A normal Claude call |
| Server-sent events | Server pushes a one-way stream | Streaming Claude's answer |
| WebSocket | Persistent, two-way, both send | Live, interactive screens |
| Polling | Client keeps asking for news | Simple, but laggy and wasteful |
| Webhook | Server calls you on an event | Notifications between systems |

ON THE EXAM

Match the pattern to the direction and frequency of the messages, not to what sounds most modern.

# Server-sent events or a websocket?

SERVER-SENT EVENTS

## One-way stream over HTTP

- Server to client only
- Ordinary HTTP, friendly to proxies
- Simple to run and to reconnect
- What the Claude API uses to stream

WEBSOCKET

## Two-way, persistent connection

- Both sides send at any time
- Upgraded from an HTTP request
- You manage heartbeats and reconnection
- For live, interactive exchanges

THE TIEBREAKER

Just watching output arrive? Server-sent events. Both sides talking continuously? A websocket.

# Where websockets fit in a Claude app

- **Not to Claude:** the Claude API is REST, and it streams with server-sent events. There is no WebSocket endpoint.
- **Your own front end and feeds:** relay streamed text to an app, take user actions back, or receive live updates.
- **Self-hosted agents:** Anthropic's hosting guide says your container may expose an HTTP or WebSocket endpoint.
- **MCP:** standard transports are stdio and Streamable HTTP; some clients also accept WebSocket servers.

THE TAKEAWAY

Websockets live between your systems and your users. Claude itself is reached over HTTP.

# Copperlet: from Claude to a tenant's phone

01
### Ask
The tenant sends a question from the app.

02
### Call
The back end calls Claude with streaming on.

03
### Parse
The SDK turns server-sent events into text.

04
### Relay
Each chunk goes to the phone over the app's websocket.

05
### Stop
If the tenant taps stop, the back end closes the stream.

THE TAKEAWAY

Two different connections, each doing the job it suits.

> Every Copperlet feature runs on Claude, and no roadmap item mentions another model provider. One enterprise customer's contract requires its calls to go through Amazon Bedrock.
>
> A tech lead proposes wrapping every Claude call in a third-party multi-provider framework, for flexibility.

## What should the team do?

A. Adopt the third-party framework, since flexibility across providers always pays off

B. Build an in-house multi-provider framework so the team controls every layer

C. Use the Anthropic SDK behind a thin internal client module that uses the Bedrock client for that customer

D. Replace the SDK with raw HTTP calls in every service to avoid any dependency at all

◆ PAUSE AND DECIDE

## What should the team do?

A Adopt the third-party framework, since flexibility across providers always pays off

*Pays a real cost for a need nobody has stated.*

B Build an in-house multi-provider framework so the team controls every layer

*The same cost, plus the team now maintains a framework.*

C Use the Anthropic SDK behind a thin internal client module that uses the Bedrock client for that customer

Meets the Bedrock contract with a client class. No framework needed.

D Replace the SDK with raw HTTP calls in every service to avoid any dependency at all

*Rebuilds retries and streaming everywhere, for no stated reason.*

THE SIGNAL

"No roadmap item" rules out the framework. "One customer on Bedrock" needs only a client class.

> An insurer is building a claims review screen. Several handlers view the same claim at once; when one adds a note, the others must see it instantly, and a Claude-drafted summary updates as notes arrive.
>
> The browser and the server both need to send messages at any time.

## Which design fits best?

A Ask Anthropic for a websocket connection straight from each browser to Claude

B Have every browser poll the server once a second for new notes and summaries

C Send each summary request to the Message Batches API and refresh the page when results arrive

D Use a websocket between browsers and the server, which calls Claude over its normal HTTP API

◆ PAUSE AND DECIDE

# Which design fits best?

A Ask Anthropic for a websocket connection straight from each browser to Claude

*No such endpoint exists, and browsers must never hold your API key.*

B Have every browser poll the server once a second for new notes and summaries

*Laggy and wasteful for constant two-way updates.*

C Send each summary request to the Message Batches API and refresh the page when results arrive

*Batch is for when nobody is waiting. Here, everyone is.*

D Use a websocket between browsers and the server, which calls Claude over its normal HTTP API

Two-way, live updates between people; HTTP to Claude.

THE SIGNAL

"Both sides send at any time" is the websocket signal. Claude stays behind your server.

> A new developer asks what the official Anthropic Python SDK will handle for them, so that they do not duplicate it in their own code.

**Which two does the SDK already do? Choose two.**

A Retry rate-limit and server errors with exponential backoff

B Validate Claude's JSON against the application's business rules

C Parse streaming events into typed events and plain text

D Choose the cheapest suitable model for each request automatically

E Store the conversation history on Anthropic's side between requests

◆ PAUSE AND DECIDE

## Which two does the SDK already do? Choose two.

A Retry rate-limit and server errors with exponential backoff

Twice by default, honouring retry-after.

B Validate Claude's JSON against the application's business rules

*Your rules, your code. The SDK cannot know them.*

C Parse streaming events into typed events and plain text

The stream helpers do exactly this.

D Choose the cheapest suitable model for each request automatically

*Model choice is yours. Module 18 covers how.*

E Store the conversation history on Anthropic's side between requests

*The API is stateless. History is your job.*

THE SIGNAL

The SDK handles transport. Business rules, model choice and state stay with you.

> A release pipeline runs in a locked-down build container where no packages may be installed. Once per release it must send Claude the changelog and get back a short summary.
>
> The container has curl and a shell, and nothing else.

## What is the best approach?

A. Call the Messages API over HTTPS with curl, setting the required headers and handling rate-limit responses

B. Install the Python SDK in the container anyway, because SDKs are always preferred

C. Open a websocket to Claude from the shell and stream the summary back

D. Skip the call, because Claude cannot be used without an SDK

◆ PAUSE AND DECIDE

## What is the best approach?

**A** Call the Messages API over HTTPS with curl, setting the required headers and handling rate-limit responses

The stated reason for raw HTTP. Take on the retry handling yourself.

**B** Install the Python SDK in the container anyway, because SDKs are always preferred

*Breaks a stated constraint. Preferred is not mandatory.*

**C** Open a websocket to Claude from the shell and stream the summary back

*The Claude API has no websocket endpoint.*

**D** Skip the call, because Claude cannot be used without an SDK

*False. Every SDK call is just HTTP underneath.*

THE SIGNAL

"No packages may be installed" is the stated reason that makes raw HTTP right.

> A web app shows Claude's answers as they are written. Today the browser asks the server for new text every two seconds, so words appear in jerky bursts and the server handles thousands of empty requests.
>
> The user only reads; nothing needs to flow back while an answer is arriving.

## What should the team change?

A Poll every quarter of a second instead, so the bursts become smaller

B Open a websocket from the browser directly to the Claude API

C Wait for the complete answer, then send it to the browser in one piece

D Stream from Claude to the server, and push the text to the browser with server-sent events

◆ PAUSE AND DECIDE

## What should the team change?

A. Poll every quarter of a second instead, so the bursts become smaller
*Eight times the wasted requests, still choppy.*

B. Open a websocket from the browser directly to the Claude API
*No such endpoint, and it would expose the API key.*

C. Wait for the complete answer, then send it to the browser in one piece
*Removes the bursts by removing streaming altogether.*

**D. Stream from Claude to the server, and push the text to the browser with server-sent events**
One-way, live, over ordinary HTTP. A natural fit.

THE SIGNAL

"The user only reads" means one-way. One-way, live updates point to server-sent events.

> A reporting job asks Claude for a long annual report with max_tokens set very high. The Python SDK raises an error before sending, saying the non-streaming request may take too long.
>
> The report genuinely needs to be long.

## What should the developer do?

A. Raise the SDK timeout to an hour and keep the request non-streaming

B. Cut max_tokens to a thousand, even though the report then gets cut short

C. Make it a streaming request and collect the final message with the SDK's stream helper

D. Bypass the SDK with raw HTTP so its check no longer applies

◆ PAUSE AND DECIDE

## What should the developer do?

A Raise the SDK timeout to an hour and keep the request non-streaming

*Ignores the guidance; long idle connections are what times out.*

B Cut max_tokens to a thousand, even though the report then gets cut short

*Breaks the requirement to fix a transport problem.*

C Make it a streaming request and collect the final message with the SDK's stream helper

Anthropic's advice for long requests. The helper still returns one message.

D Bypass the SDK with raw HTTP so its check no longer applies

*Removes the warning, not the timeout it warns about.*

THE SIGNAL

Long output means stream it. The SDK's warning is advice, not an obstacle.

# Module 4 in five lines

- **An SDK wraps plain HTTP:** auth, retries with backoff, timeouts, streaming, types, platform clients.
- **Tune the SDK, do not replace it.** Raw HTTP needs a stated reason.
- **Abstraction is a cost.** Pay it for a real requirement; a thin internal seam covers most needs.
- **One-way live output:** server-sent events. **Both sides talking:** a websocket.
- **Claude is reached over HTTP.** Websockets belong between your systems and your users.

COMING UP

Next: prompt engineering for developers.